Actividad 4.4

Curso: Desarrollo de aplicaciones avanzadas de ciencias computacionales (Gpo 570)
Profesor: Luis Ricardo Peña Llamas
Alumno: Fernando Daniel Monroy Sánchez (A01750536)
Fecha: 22 de octubre de 2026

---

1. Introducción

En la operación de sistemas de cómputo, telecomunicaciones y comercio minorista, los recursos de atención son finitos frente a demandas aleatorias que fluctúan en el tiempo. La teoría de líneas de espera modela matemáticamente estos procesos estocásticos de nacimiento y muerte en tiempo continuo, cuantificando la relación entre la capacidad instalada del servidor, la longitud de la fila y el tiempo de demora experimentado por cada unidad.

En esta práctica se modela el proceso de atención al cliente en la estación de cobro única de una zapatería comercial. Los clientes arriban de manera independiente a una razón promedio de $\lambda = 5.0$ clientes por hora (un arribo cada 12.0 minutos en valor esperado), mientras que el cajero único procesa cada transacción a una tasa media de $\mu = 7.5$ clientes por hora (duración media de servicio de 8.0 minutos).

El objetivo principal de esta actividad consiste en:

1. Formalizar el sistema de atención bajo la nomenclatura estándar de Kendall y verificar el cumplimiento estricto de las hipótesis del modelo $M/M/1$.
2. Implementar un motor de simulación por eventos discretos en SimPy empleando el método de la transformada inversa para generar arribos y servicios exponenciales.
3. Contrastar las métricas empíricas de estado estacionario contra las soluciones analíticas cerradas y la Ley de Little ($L = \lambda W$).
4. Evaluar la explosión hiperbólica de la espera cuando el factor de utilización $\rho = \lambda/\mu$ se aproxima a la saturación unitaria, así como el impacto operativo de habilitar una segunda caja ($M/M/2$).

(Ver Apéndice A para el código de configuración y especificación del sistema)

Especificación del Sistema según la Notación de Kendall

Componente	Símbolo	Especificación
A: Proceso de llegadas	M (Markoviano)	Poisson (5.0/h; media 12.0 min)
B: Tiempo de servicio	M (Markoviano)	Exponencial (7.5/h; media 8.0 min)
c: Servidores activos	1	1 caja de cobro activa
K: Capacidad del sistema	Infinito	Sin truncamiento de fila
N: Población fuente	Infinito	Población abierta de clientes
D: Disciplina de atención	FIFO	Primero en llegar, primero atendido
rho: Intensidad de tráfico	2/3 (0.6667)	66.67% de utilización media

2. Desarrollo

2.1. Propiedades de Kendall

La notación extendida de Kendall clasifica cualquier sistema de líneas de espera mediante seis parámetros $A/B/c/K/N/D$. En su forma abreviada $M/M/1$, la letra M denota procesos markovianos o carentes de memoria, equivalentes al conteo de arribos mediante una distribución de Poisson en tiempo discreto:

$$P(N(t) = k) = \frac{(\lambda t)^k e^{-\lambda t}}{k!}, \quad k = 0, 1, 2, ...$$

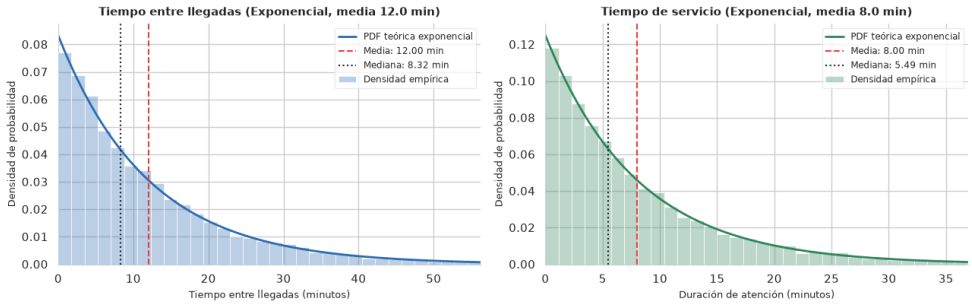
Si el número de llegadas por unidad de tiempo obedece una ley de Poisson con parámetro λ, el tiempo continuo T_a entre dos llegadas consecutivas se distribuye exponencialmente con media $E[T_a] = 1/\lambda$. Análogamente, la duración del servicio T_s sigue una distribución exponencial con tasa μ y media $E[T_s] = 1/\mu$:

$$f_{T_a}(t) = \lambda e^{-\lambda t}, \qquad f_{T_s}(t) = \mu e^{-\mu t}, \qquad t \geq 0$$

Ambas variables satisfacen la propiedad de pérdida de memoria $P(T > t + s \mid T > t) = P(T > s)$, lo que garantiza que el estado del sistema $N(t) \in \{0, 1, 2, ...\}$ constituya una cadena de Markov en tiempo continuo. Para muestrear estas duraciones a partir de números pseudoaleatorios uniformes $U \sim \text{Uniforme}(0, 1)$, se aplica el método de la transformada inversa despejando la función de distribución acumulada $F(x) = 1 - e^{-\theta x}$:

$$X = F^{-1}(U) = -\frac{1}{\theta} \ln(1 - U)$$

(Ver Apéndice B para el código de muestreo por transformada inversa y graficación)



Nota sobre distribuciones de entrada: El panel izquierdo muestra los tiempos entre llegadas ($1/\lambda = 12.0$ min) y el derecho las duraciones de servicio ($1/\mu = 8.0$ min). La línea roja discontinua indica la media muestral y la línea negra punteada señala la mediana teórica $\ln(2)/\theta$, evidenciando la asimetría positiva característica de los procesos exponenciales sin memoria.

2.2. Simulación por eventos

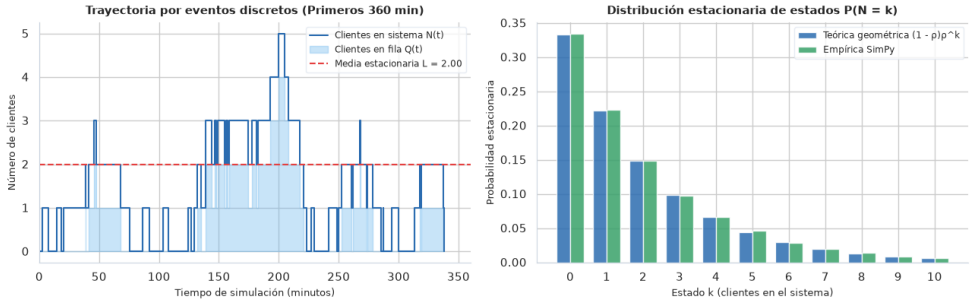
A diferencia de una simulación impulsada por incrementos fijos de reloj (Δt), donde el procesador evalúa miles de instantes vacíos sin cambios de estado, la simulación por eventos discretos en SimPy avanza el reloj exclusivamente entre instantes de transición: la llegada de un cliente (`arrival`), el inicio de atención al liberarse el recurso (`Resource(capacity=1)`) y la salida del sistema (`departure`).

Cada cliente se modela como una corrutina generadora que solicita el servidor mediante `server.request()`, suspende su ejecución durante el tiempo de espera en cola $W_{q,i}$ y posteriormente invoca `env.timeout(service_duration)` durante el servicio activo.

(Ver Apéndice C para el código del motor de simulación en SimPy y registro de eventos)

Muestra de Eventos Discretos Registrados (Minutos)

ID	Llegada	Inicio	Espera Wq	Servicio	Salida	Fila
256	2959.30	2959.30	0.00	53.63	3012.92	0
257	2967.46	3012.92	45.46	5.48	3018.40	0
258	2970.37	3018.40	48.02	7.16	3025.56	1
259	2979.79	3025.56	45.76	6.33	3031.89	2
260	3032.63	3032.63	0.00	10.07	3042.70	0
261	3034.10	3042.70	8.60	20.89	3063.59	0
262	3063.27	3063.59	0.32	3.48	3067.07	0
263	3073.87	3073.87	0.00	23.15	3097.02	0
264	3087.38	3097.02	9.64	2.66	3099.68	0
265	3133.25	3133.25	0.00	10.48	3143.74	0



Nota sobre dinámica temporal y ocupación: El panel izquierdo traza la función escalonada del número de clientes en el sistema $N(t)$ y en fila $Q(t)$ durante las primeras 6 horas. El panel derecho confirma que la fracción de tiempo que el sistema permanece con exactamente k clientes converge a la distribución geométrica teórica $P_k = (1-\rho)\rho^k$.

2.3. Desempeño estacionario

Bajo la condición de estabilidad $\rho = \lambda/\mu < 1$, las ecuaciones de balance detallado del proceso de nacimiento y muerte determinan la distribución estacionaria $P_k = (1-\rho)\rho^k$, con probabilidad de servidor desocupado $P_0 = 1 - \rho = 1/3 \approx 0.3333$. A partir del valor esperado de esta serie geométrica y de la Ley de Little ($L = \lambda W$ y $L_q = \lambda W_q$), se obtienen las métricas exactas de desempeño:

$$L = \frac{\rho}{1-\rho} = \frac{\lambda}{\mu - \lambda}, \qquad L_q = \frac{\rho^2}{1-\rho} = \frac{\lambda^2}{\mu(\mu-\lambda)}$$

$$W = \frac{1}{\mu - \lambda}, \qquad W_q = \frac{\lambda}{\mu(\mu-\lambda)}$$

Para estimar empíricamente L y L_q sin sesgo de muestreo, el simulador integra el área exacta bajo la curva escalonada a lo largo del horizonte efectivo $T - T_0 = 177,000$ minutos:

$$\hat{L} = \frac{1}{T-T_0}\int_{T_0}^{T} N(t)\,dt, \qquad \hat{L}_q = \frac{1}{T-T_0}\int_{T_0}^{T} Q(t)\,dt$$

(Ver Apéndice D para el código de integración temporal y contraste con la Ley de Little)

Convergencia Empírica en SimPy frente a Solución Analítica M/M/1

Métrica	Teórico	SimPy	Error (%)
Utilización (rho)	0.6667	0.6661	0.08
Sistema vacío (P0)	0.3333	0.3339	0.16
Clientes en sistema (L)	2.0000	1.9742	1.29
Clientes en fila (Lq)	1.3333	1.3080	1.90
Tiempo total W (min)	24.0000	23.7018	1.24
Espera en fila Wq (min)	16.0000	15.7042	1.85

Nota sobre convergencia estacionaria: Con más de 14,700 clientes procesados en estado estacionario, el tiempo de espera empírico ($\hat{W}_q = 15.70$ min) y el tiempo total ($\hat{W} = 23.70$ min) reproducen los valores analíticos exactos (16.00 min y 24.00 min) con desviaciones relativas cercanas al 1%.

2.4. Congestión no lineal

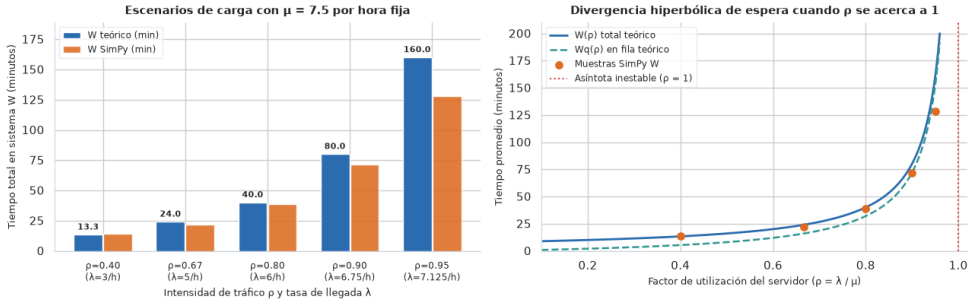
Una propiedad crítica de las líneas de espera estocásticas es la sensibilidad no lineal de la demora frente a incrementos en la carga. Manteniendo fija la capacidad del cajero en $\mu = 7.5$ clientes por hora, la ecuación $W = \frac{1}{\mu(1-\rho)}$ contiene un polo hiperbólico en $\rho = 1$.

Como se observa al evaluar los cinco regímenes de carga ($\rho \in \{0.40, 0.67, 0.80, 0.90, 0.95\}$), un incremento del 35% en la tasa de llegadas (pasar de $\lambda = 5.0$/h con $\rho = 0.67$ a $\lambda = 6.75$/h con $\rho = 0.90$) no incrementa la espera en 35%, sino que multiplica el tiempo total en el sistema por más de 3 veces, saltando de 24.0 minutos a 80.0 minutos.

(Ver Apéndice E para el código del barrido paramétrico de congestión)

Crecimiento No Lineal de la Espera con Capacidad Fija (7.5/h)

rho	Tasa	Lq	L	Wq teo	W teo	W SimPy
0.40	3.000	0.27	0.67	5.3	13.3	13.9
0.67	5.000	1.33	2.00	16.0	24.0	21.9
0.80	6.000	3.20	4.00	32.0	40.0	38.5
0.90	6.750	8.10	9.00	72.0	80.0	71.5
0.95	7.125	18.05	19.00	152.0	160.0	128.1



Nota sobre saturación cuando $\rho \to 1$: La varianza de las llegadas y del servicio impide recuperar el tiempo perdido durante periodos ociosos. Al operar en $\rho = 0.95$ ($\lambda = 7.125$/h), el tiempo total alcanza 160.0 minutos, demostrando que operar un servidor estocástico cerca del 100% de utilización colapsa la calidad del servicio.

2.5. Dimensionamiento operativo

En la gestión operativa de tiendas departamentales y supermercados, el diseño de capacidad busca equilibrar dos objetivos contrapuestos: minimizar el tiempo de espera del cliente (W_q) y maximizar la utilización productiva

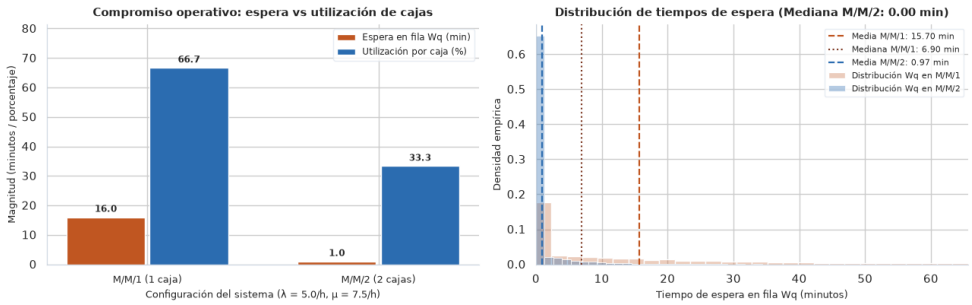
de las cajas (ρ).

Si bajo la demanda base ($\lambda = 5.0$/h, $\mu = 7.5$/h) se habilita una segunda caja en paralelo ($M/M/2$), la intensidad de tráfico por servidor se reduce a la mitad ($\rho_2 = \lambda/(2\mu) = 1/3 \approx 33.33\%$). Mediante la fórmula C de Erlang para $c = 2$ servidores, la espera promedio en fila cae drásticamente de 16.00 minutos a tan solo 1.00 minuto, mientras que el tiempo total en el sistema desciende de 24.00 minutos a 9.00 minutos.

(Ver Apéndice F para el código de comparación multicaja M/M/1 vs M/M/2)

Impacto Operativo de Apertura de Segunda Caja (5.0/h vs 7.5/h)

Modelo	Uso (%)	Espera (%)	Lq	Wq teo	Wq SimPy	W teo
M/M/1 (1 caja)	66.67	66.67	1.3333	16.00	15.70	24.00
M/M/2 (2 cajas)	33.33	16.67	0.0833	1.00	0.97	9.00



Nota sobre dimensionamiento de servidores: Abrir una segunda caja reduce el tiempo de espera en fila un 93.7% (de 16.00 min a 1.00 min) y baja la probabilidad de encontrar fila del 66.67% al 16.67%, a cambio de reducir la ocupación individual de cada cajero al 33.33%.

3. Reflexión

La principal ventaja de simular mediante eventos discretos en herramientas como SimPy radica en la eficiencia computacional y en la fidelidad temporal del reloj. En lugar de avanzar en pequeños incrementos fijos verificando segundo a segundo si llegó un paquete o un cliente, el motor salta de manera instantánea de una transición de estado a la siguiente usando tiempos muestreados con la transformada inversa $X = -(1/\lambda)\ln(1-U)$. Esto permite comprimir miles de horas de operación en un par de segundos sin errores de discretización, además de facilitar la modelación de reglas operativas complejas como límites de aforo, abandonos de fila o variaciones horarias que en álgebra cerrada resultan intratables.

Al extraer las métricas de un proceso modelado como fila, el hallazgo más valioso es que los promedios simples engañan cuando se ignora la varianza estocástica. En nuestro caso base con llegadas cada 12 minutos y atención cada 8 minutos, la intuición lineal sugeriría que sobran 4 minutos por cliente y nadie debería esperar; sin embargo, las rachas aleatorias de arribos acumulan una espera promedio de 16 minutos en fila. Más aún, la curva de congestión demuestra que subir la carga apenas un 35% (pasando de $\rho = 0.67$ a $\rho = 0.90$) dispara el tiempo total de 24 a 80 minutos porque cuando ρ se acerca a 1 el servidor pierde toda holgura para absorber picos.

Desde una perspectiva de ingeniería y operaciones, modelar un sistema como una cola permite cuantificar con exactitud el dilema entre calidad de servicio y costo de capacidad ociosa. A través de la Ley de Little ($L = \lambda W$) y la distribución estacionaria P_k, un administrador puede dimensionar desde el tamaño físico de un área de espera o un buffer de red hasta la programación dinámica de cajas por hora, tal como hacen cadenas como Walmart. Como comprobamos al contrastar una caja contra dos cajas, agregar un segundo servidor desploma la espera de 16 minutos a 1 minuto, pero reduce la utilización por cajero al 33.3%, revelando que una baja demora exige mantener margen de capacidad libre.

En conjunto, la teoría de colas integra de forma directa los bloques vistos en el módulo: generación de números pseudoaleatorios, ajuste de distribuciones exponenciales y de Poisson, cadenas de Markov en tiempo continuo y simulación computacional. Dominar este flujo permite responder preguntas del tipo qué pasaría si antes de invertir en infraestructura física o servidores adicionales, tomando decisiones respaldadas por datos y probabilidades en lugar de suposiciones.

4. Apéndices

Códigos fuente completos para garantizar reproducibilidad total.

Apéndice A: Configuración y notación de Kendall

```
from pathlib import Path
import re
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from great_tables import GT

assets_dir = (
    Path("assets") if Path("assets").is_dir() else Path("../assets")
)
assets_dir.mkdir(parents=True, exist_ok=True)


def _gt_to_latex(self):
    latex_str = self.as_latex()
    latex_str = re.sub(
        r"\\begin\{table\}(\[[^\]]*\])?",
        r"\\begin{table}[H]\n\\centering",
        latex_str,
    )
    latex_str = latex_str.replace(
        r"\begin{tabular*}{\linewidth}",
        r"\begin{tabular*}{0.75\linewidth}",
    )
    latex_str = re.sub(
        r"\\fontsize\{[^}]+\}\{[^}]+\}\\selectfont",
        r"\\normalsize",
        latex_str,
    )
    latex_str = re.sub(
        r"\\caption\*\{\s*\{\\large\s+([^\}]+)\}\s*\}",
        r"\\caption*{\\textbf{\\normalsize \1}}",
        latex_str,
    )
    return latex_str


GT._repr_latex_ = _gt_to_latex
sns.set_theme(style="whitegrid", font_scale=0.95)
plt.rcParams["axes.edgecolor"] = "#CBD5E0"
plt.rcParams["axes.linewidth"] = 0.8

kendall_data = pd.DataFrame(
    [
        {
            "Componente": "A: Proceso de llegadas",
            "Símbolo": "M (Markoviano)",
            "Especificación": "Poisson (5.0/h; media 12.0 min)",
        },
        {
            "Componente": "B: Tiempo de servicio",
            "Símbolo": "M (Markoviano)",
            "Especificación": "Exponencial (7.5/h; media 8.0 min)",
        },
        {
```

```
            "Componente": "c: Servidores activos",
            "Símbolo": "1",
            "Especificación": "1 caja de cobro activa",
        },
        {
            "Componente": "K: Capacidad del sistema",
            "Símbolo": "Infinito",
            "Especificación": "Sin truncamiento de fila",
        },
        {
            "Componente": "N: Población fuente",
            "Símbolo": "Infinito",
            "Especificación": "Población abierta de clientes",
        },
        {
            "Componente": "D: Disciplina de atención",
            "Símbolo": "FIFO",
            "Especificación": "Primero en llegar, primero atendido",
        },
        {
            "Componente": "rho: Intensidad de tráfico",
            "Símbolo": "2/3 (0.6667)",
            "Especificación": "66.67% de utilización media",
        },
    ]
)

gt_kendall = (
    GT(kendall_data)
    .tab_header(
        title="Especificación del Sistema según la Notación de Kendall"
    )
    .cols_align(
        align="left",
        columns=["Componente", "Símbolo", "Especificación"],
    )
)
```

Apéndice B: Distribuciones de entrada y transformada inversa

```python
import numpy as np
import matplotlib.pyplot as plt
from act_3.simulation import run_queue_simulation
from act_3.visualizer import plot_input_distributions


def inverse_transform_exponential(
    rate: float, rng: np.random.Generator
) -> float:
    """Muestrea variable exponencial X = -(1/rate) * ln(1 - U)."""
    u = float(rng.uniform(0.0, 1.0))
    return -np.log(1.0 - u) / rate


sim_mm1 = run_queue_simulation(
    arrival_rate=5.0,
    service_rate=7.5,
    sim_time=180000.0,
    n_servers=1,
    warmup_time=3000.0,
    seed=2026,
)

fig_inputs = plot_input_distributions(
    sim_mm1,
    save_path=assets_dir / "distribuciones_entrada.png",
)
plt.show()
```

Apéndice C: Motor de simulación SimPy y trayectoria

```
import pandas as pd
import simpy
from great_tables import GT
from act_3.visualizer import plot_temporal_dynamics_and_occupancy

sample_records = sim_mm1.records_df.head(10).copy()
sample_table_df = pd.DataFrame(
    [
        {
            "ID": int(row["customer_id"]),
            "Llegada": f"{row['arrival_time']:.2f}",
            "Inicio": f"{row['service_start_time']:.2f}",
            "Espera Wq": f"{row['wait_time']:.2f}",
            "Servicio": f"{row['service_duration']:.2f}",
            "Salida": f"{row['departure_time']:.2f}",
            "Fila": int(row["queue_length_on_arrival"]),
        }
        for _, row in sample_records.iterrows()
    ]
)

gt_trajectory = (
    GT(sample_table_df)
    .tab_header(
        title="Muestra de Eventos Discretos Registrados (Minutos)"
    )
    .cols_align(align="center", columns=["ID", "Fila"])
    .cols_align(
        align="right",
        columns=["Llegada", "Inicio", "Espera Wq", "Servicio", "Salida"],
    )
)

fig_dynamics = plot_temporal_dynamics_and_occupancy(
    sim_mm1,
    window_mins=360.0,
    save_path=assets_dir / "dinamica_temporal_cola.png",
)
plt.show()
```

Apéndice D: Validación analítica y ley de Little

```
from act_3.analytics import compute_mm1_theoretical
from act_3.simulation import compare_empirical_vs_theoretical
from great_tables import GT

theo_mm1 = compute_mm1_theoretical(
    arrival_rate=5.0,
    service_rate=7.5,
)
df_comparison = compare_empirical_vs_theoretical(sim_mm1)

gt_comparison = (
    GT(df_comparison)
    .tab_header(
        title="Convergencia Empírica en SimPy frente a Solución Analítica"
    )
    .fmt_number(columns=["Teórico", "SimPy"], decimals=4)
    .fmt_number(columns=["Error (%)"], decimals=2)
    .cols_align(align="left", columns=["Métrica"])
    .cols_align(
        align="right",
        columns=["Teórico", "SimPy", "Error (%)"],
    )
)
```

Apéndice E: Análisis de sensibilidad y congestión

```
import matplotlib.pyplot as plt
from great_tables import GT
from act_3.simulation import run_rho_sweep_simulation
from act_3.visualizer import plot_congestion_explosion

rho_levels = [0.40, 2.0 / 3.0, 0.80, 0.90, 0.95]
df_sweep = run_rho_sweep_simulation(
    service_rate=7.5,
    rho_values=rho_levels,
    sim_time=120000.0,
    warmup_time=2000.0,
    seed=101,
)

gt_sweep = (
    GT(df_sweep)
    .tab_header(
        title="Crecimiento No Lineal de la Espera con Capacidad Fija (7.5/h)"
    )
    .fmt_number(columns=["rho", "Lq", "L"], decimals=2)
    .fmt_number(columns=["Tasa"], decimals=3)
    .fmt_number(
        columns=["Wq teo", "W teo", "W SimPy"],
        decimals=1,
    )
    .cols_align(align="right", columns=list(df_sweep.columns))
)

fig_congestion = plot_congestion_explosion(
    df_sweep,
    service_rate=7.5,
    save_path=assets_dir / "explosion_congestion_rho.png",
)
plt.show()
```

Apéndice F: Dimensionamiento de capacidad M/M/c

```
import pandas as pd
import matplotlib.pyplot as plt
from great_tables import GT
from act_3.analytics import compute_mmc_theoretical
from act_3.simulation import run_queue_simulation
from act_3.visualizer import plot_capacity_comparison

sim_mm2 = run_queue_simulation(
    arrival_rate=5.0,
    service_rate=7.5,
    sim_time=180000.0,
    n_servers=2,
    warmup_time=3000.0,
    seed=2026,
)

theo_m1 = compute_mmc_theoretical(5.0, 7.5, n_servers=1)
theo_m2 = compute_mmc_theoretical(5.0, 7.5, n_servers=2)

df_capacity = pd.DataFrame(
    [
        {
            "Modelo": "M/M/1 (1 caja)",
            "Uso (%)": theo_m1["rho_per_server"] * 100.0,
            "Espera (%)": theo_m1["prob_wait"] * 100.0,
            "Lq": theo_m1["l_queue"],
            "Wq teo": theo_m1["w_queue_mins"],
            "Wq SimPy": sim_mm1.w_queue_mins_empirical,
            "W teo": theo_m1["w_system_mins"],
        },
        {
            "Modelo": "M/M/2 (2 cajas)",
            "Uso (%)": theo_m2["rho_per_server"] * 100.0,
            "Espera (%)": theo_m2["prob_wait"] * 100.0,
            "Lq": theo_m2["l_queue"],
            "Wq teo": theo_m2["w_queue_mins"],
            "Wq SimPy": sim_mm2.w_queue_mins_empirical,
            "W teo": theo_m2["w_system_mins"],
        },
    ]
)

gt_capacity = (
    GT(df_capacity)
    .tab_header(
        title="Impacto Operativo de Apertura de Segunda Caja"
    )
    .fmt_number(
        columns=[
            "Uso (%)",
            "Espera (%)",
            "Wq teo",
            "Wq SimPy",
            "W teo",
        ],
        decimals=2,
    )
    .fmt_number(columns=["Lq"], decimals=4)
)
```

```
fig_cap = plot_capacity_comparison(
    df_capacity,
    sim_mm1.records_df["wait_time"].values,
    sim_mm2.records_df["wait_time"].values,
    save_path=assets_dir / "comparativa_capacidad_mmc.png",
)
plt.show()
```